Trabajo de Campo - REAGRO - Grupo 12

S31

Integrantes: DOMATO TOBIAS, GILES GUILLERMO, MARTINEZ BALTAZAR, OLIVAN JEREMÍAS, PUERTAS BAUTISTA

Descripción del Proyecto: REAGRO (Red de Gestión y Distribución de Excedentes Frutihortícolas)

1. Contexto y Problemática

El proyecto se enmarca en las actividades del Banco de Alimentos de La Plata (BALP), entidad que articula el rescate de excedentes del cordón frutihortícola con más de 200 productores para abastecer a unos 150 comedores y entidades comunitarias. Actualmente, este circuito opera de forma manual y fragmentada mediante planillas que alimentan de manera desconectada un software de georreferenciación (QGIS), en una región donde se desperdician 10.000 toneladas anuales de alimentos. Esta dinámica presenta severas limitaciones:

- **Falta de trazabilidad:** No se registran de forma centralizada el origen, tipo de producto, motivo de donación, vida útil ni destino final de los lotes.
- **Distribución ineficiente:** Las entregas dependen de la disponibilidad del día previo y no de un criterio objetivo de necesidad o equidad social.
- **Sobrecostos logísticos:** Todo excedente pasa obligatoriamente por el depósito central, impidiendo entregas directas entre actores cercanos.
- **Falta de coordinación agrícola:** El desconocimiento de la siembra colectiva genera sobreoferta estructural y pérdidas masivas de cosechas.

2. Solución Propuesta

Se propone el desarrollo de REAGRO, una plataforma digital en formato Progressive Web App (PWA) para dispositivos móviles y de escritorio, con capacidad de trabajo offline y sincronización diferida para zonas rurales. Su propósito es centralizar la gestión operativa integrando dos ejes: la trazabilidad logística de los excedentes en tiempo real y la planificación comunitaria de siembra para prevenir la sobreproducción futura.

3. Alcance y Funcionalidades Principales

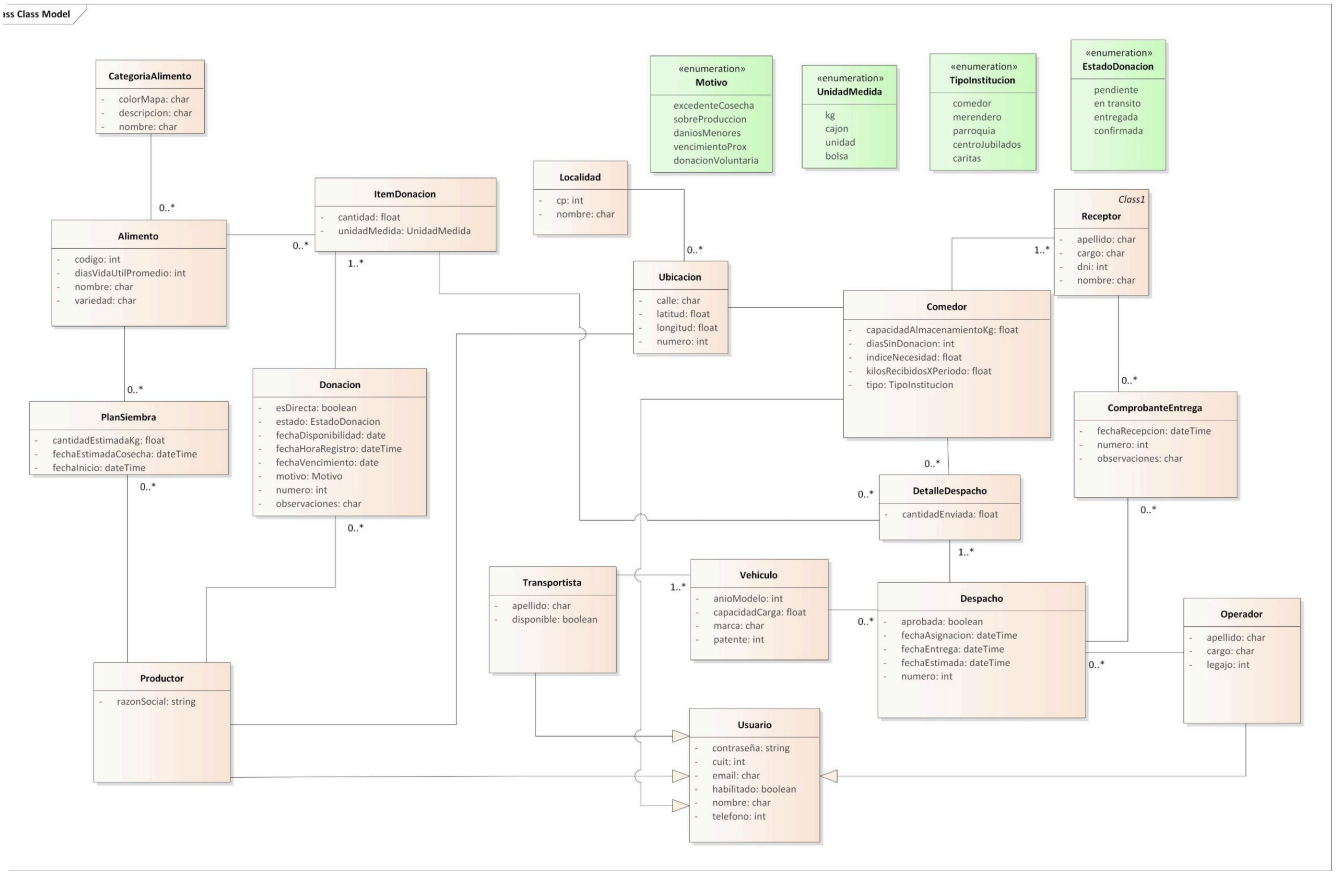
REAGRO cubre el circuito integral de donaciones conectando a productores, transportistas, comedores y al equipo coordinador del BALP:

- **Gestión y Trazabilidad de Excedentes:** Registro ágil de donaciones (alimento, variedad, volumen en kg/cajones, vencimiento, ubicación GPS y motivo) y control de su ciclo de vida (Pendiente, En Tránsito, Entregada, Confirmada).
- **Distribución Inteligente y Ruteo:** Cálculo automático del índice de necesidad de cada comedor (días sin asistencia y kilos recibidos) para sugerir rutas óptimas al transportista, siempre bajo validación del BALP.
- **Canal de Donación Directa:** Conexión directa entre productor y comedor de cercanía para lotes menores, evitando el paso por el depósito central.
- **Comunidad de Siembra:** Módulo anónimo entre productores para cargar intenciones de siembra y generar alertas de sobreoferta proyectada para entidades de control (BALP, UNLP y Municipalidad).
- **Gestión y Auditoría de Usuarios:** Flujo de solicitud, verificación y aprobación de altas de productores, entidades y transportistas administrado por el BALP.

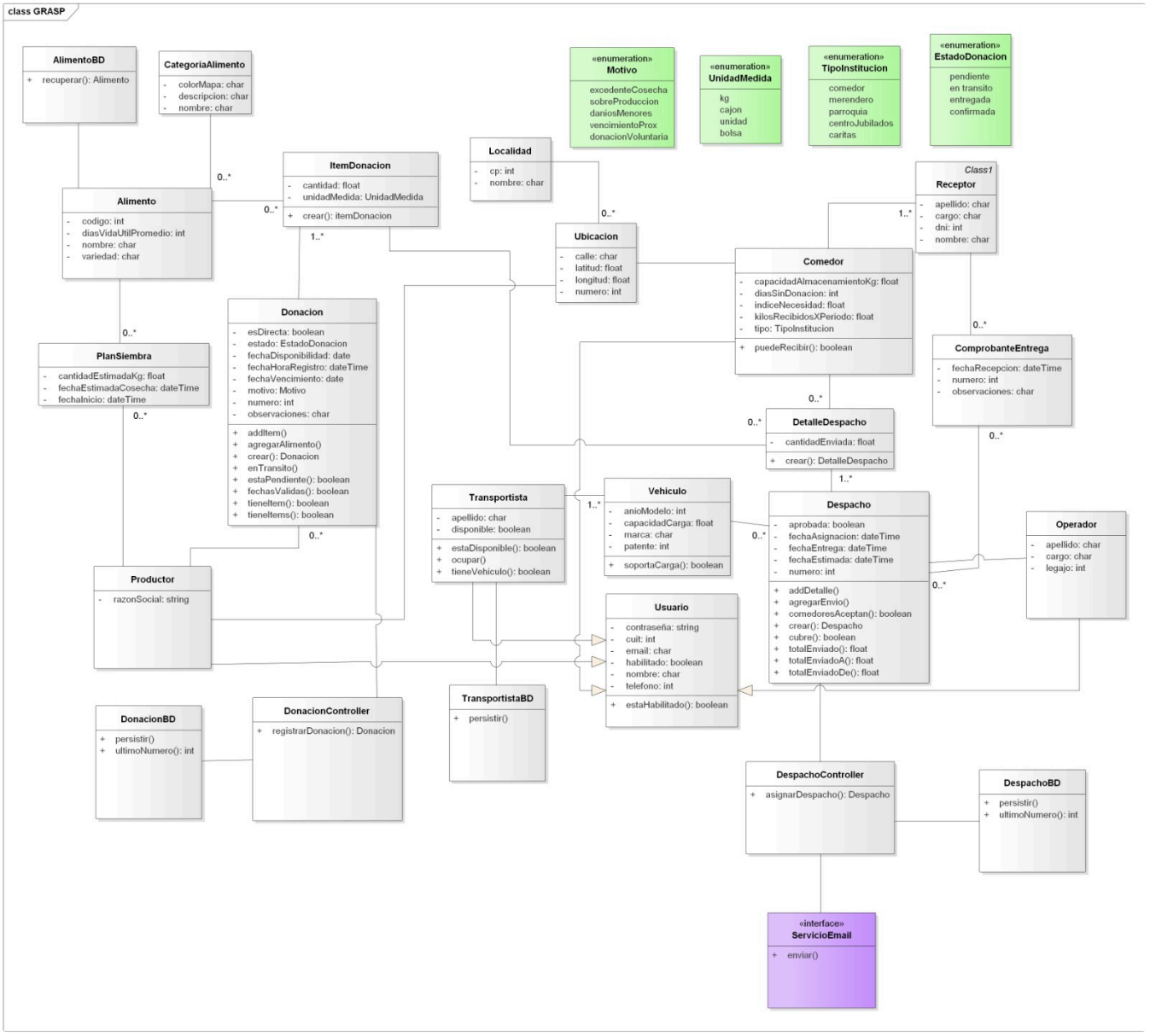
4. Objetivos del Negocio

El sistema busca reducir los tiempos de rescate de alimentos perecederos y minimizar el desperdicio en la región, optimizando los costos de transporte y garantizando una distribución equitativa basada en la vulnerabilidad social real, con métricas claras y en tiempo real para la toma de decisiones.

Modelo de Dominio



GRASP



Registrar Donación

Requerimiento 1: “El sistema deberá permitir registrar una donación”.

```
DonacionController :: registrarDonacion(
    productor: Productor,
    motivo: Motivo,
    fechaDisponibilidad: date,
    fechaVencimiento: date,
    observaciones: char,
    codigosAlimento[1..*]: int,
    cantidades[1..*]: float,
    unidadesMedida[1..*]: UnidadMedida
) {
```

```
    If productor.estaHabilitado() then
        donacion = Donacion.crear(productor, motivo, fechaDisponibilidad,
                                  fechaVencimiento, observaciones)

        for i = 1 to codigosAlimento.size()
            If cantidades[i] > 0 then
                alimento = AlimentoBD.recuperar(codigosAlimento[i])
                donacion.agregarAlimento(alimento, cantidades[i],
unidadesMedida[i])

        If donacion.tieneItems() and donacion.fechasValidas() then
            DonacionBD.persistir(donacion)
}

Donacion :: crear(
    productor: Productor,
    motivo: Motivo,
    fechaDisponibilidad: date,
    fechaVencimiento: date,
    observaciones: char
) : Donacion {
    donacion = self.new()
    donacion.setNumero(DonacionBD.ultimoNumero() + 1)
    donacion.setProductor(productor)
    donacion.setFechaHoraRegistro(getDate())
    donacion.setMotivo(motivo)
    donacion.setFechaDisponibilidad(fechaDisponibilidad)
    donacion.setFechaVencimiento(fechaVencimiento)
    donacion.setObservaciones(observaciones)
    donacion.setEsDirecta(false)
    donacion.setEstado(EstadoDonacion.pendiente)
    return donacion
}

ItemDonacion :: crear(alimento: Alimento, cantidad: float, unidadMedida:
UnidadMedida) : ItemDonacion {
    item = self.new()
    item.setAlimento(alimento)
    item.setCantidad(cantidad)
    item.setUnidadMedida(unidadMedida)
    return item
}

Donacion :: agregarAlimento(alimento: Alimento, cantidad: float,
unidadMedida: UnidadMedida) {
    item = ItemDonacion.crear(alimento, cantidad, unidadMedida)
```

```
    self.addItem(item)
}

Donacion :: addItem(i: ItemDonacion) {
    self.items->add(i)
    i.setDonacion(self)
}

// Validaciones
Usuario :: estaHabilitado() : boolean {
    return self.getHabilitado()
}

Donacion :: tieneItems() : boolean {
    return self.items->size() > 0
}

Donacion :: fechasValidas() : boolean {
    return self.fechaVencimiento > self.fechaDisponibilidad
}
```

Asignar Despacho

Requerimiento 2: “El sistema deberá permitir asignar un despacho a una donación”.

```
DespachoController :: asignarDespacho(
    donacion: Donacion,
    transportista: Transportista,
    vehiculo: Vehiculo,
    operador: Operador,
    fechaEstimada: date,
    items[1..*]: ItemDonacion,
    comedores[1..*]: Comedor,
    cantidades[1..*]: float
) {
    If donacion.estaPendiente() and transportista.estaDisponible()
       and transportista.tieneVehiculo(vehiculo) then

        despacho = Despacho.crear(operador, vehiculo, fechaEstimada)

        for i = 1 to items.size()
            If cantidades[i] > 0 and donacion.tieneItem(items[i]) then
                despacho.agregarEnvio(items[i], comedores[i], cantidades[i])

        If vehiculo.soportaCarga(despacho.totalEnviado()) and
```

```
despacho.cubre(donacion)
           and despacho.comedoresAceptan() then

            donacion.enTransito()
            transportista.ocupar()
            DespachoBD.persistir(despacho)
            DonacionBD.persistir(donacion)
            TransportistaBD.persistir(transportista)
            ServicioEmail.enviar(transportista.getEmail(), 'Despacho
asignado',
                'Se le asignó el despacho ' + despacho.getNumero() +
                ' por ' + despacho.totalEnviado() + ' kg')
}

Despacho :: crear(operador: Operador, vehiculo: Vehiculo, fechaEstimada:
date) : Despacho {
    despacho = self.new()
    despacho.setNumero(DespachoBD.ultimoNumero() + 1)
    despacho.setOperador(operador)
    despacho.setVehiculo(vehiculo)
    despacho.setFechaAsignacion(getDate())
    despacho.setFechaEstimada(fechaEstimada)
    despacho.setAprobada(true)
    return despacho
}

DetalleDespacho :: crear(item: ItemDonacion, comedor: Comedor,
cantidadEnviada: float) : DetalleDespacho {
    detalle = self.new()
    detalle.setItemDonacion(item)
    detalle.setComedor(comedor)
    detalle.setCantidadEnviada(cantidadEnviada)
    return detalle
}

Despacho :: agregarEnvio(item: ItemDonacion, comedor: Comedor,
cantidadEnviada: float) {
    detalle = DetalleDespacho.crear(item, comedor, cantidadEnviada)
    self.addDetalle(detalle)
}

Despacho :: addDetalle(d: DetalleDespacho) {
    self.detalles->add(d)
    d.setDespacho(self)
}
```

```
// Totales y validaciones del despacho
Despacho :: totalEnviado() : float {
    return self.detalles->collect(d | d.getCantidadEnviada())->sum()
}

Despacho :: totalEnviadoA(c: Comedor) : float {
    return self.detalles->select(d | d.getComedor() = c)
                        ->collect(d | d.getCantidadEnviada())->sum()
}

Despacho :: totalEnviadoDe(i: ItemDonacion) : float {
    return self.detalles->select(d | d.getItemDonacion() = i)
                        ->collect(d | d.getCantidadEnviada())->sum()
}

Despacho :: cubre(d: Donacion) : boolean {
    for i = 1 to d.getItems().size()
        If self.totalEnviadoDe(d.getItems()[i]) <> d.getItems()
[i].getCantidad() then
            return false
    return true
}

Despacho :: comedoresAceptan() : boolean {
    for i = 1 to self.detalles->size()
        c = self.detalles[i].getComedor()
        If not c.puedeRecibir(self.totalEnviadoA(c)) then
            return false
    return true
}

// Consultas y cambios de estado en las demás clases
Donacion :: estaPendiente() : boolean {
    return self.estado = EstadoDonacion.pendiente
}

Donacion :: tieneItem(i: ItemDonacion) : boolean {
    return self.items->includes(i)
}

Donacion :: enTransito() {
    self.setEstado(EstadoDonacion.enTransito)
}

Transportista :: estaDisponible() : boolean {
    return self.getDisponible()
```

```
}

Transportista :: tieneVehiculo(v: Vehiculo) : boolean {
    return self.vehiculos->includes(v)
}

Transportista :: ocupar() {
    self.setDisponible(false)
}

Vehiculo :: soportaCarga(kg: float) : boolean {
    return kg <= self.capacidadCarga
}

Comedor :: puedeRecibir(kg: float) : boolean {
    return kg <= self.capacidadAlmacenamientoKg
}
```

Persistencia

